Beyond BusyLean's Contender 6

Autonomous Model Ramification and a
46-fold Tower-Feedback Separation

Research proposal prepared for Vladimir Reshetnikov
October 12, 2026

Abstract

We propose a closed, computable Lean definition that strengthens the current BusyLean contender by changing what the object language can interpret, not merely by enlarging its numerical argument. A fixed seven-slot interface makes lower reflection operators, forest recursors, and program-generated tree schedules available to higher interpretations. The interface closes under all finite model ramifications, and is then ramified structurally along a tree produced by the strengthened language itself. A depth-11 program provides a uniform comparison certificate, valid for every external oracle. If C_6 is the adopted contender at the pinned repository revision, and $\Theta(x) = (y \mapsto 2^y)^{[x+1]}(x+1)$, the construction satisfies

$$\boxed{\Theta^{[46]}(C_6) < C_7.}$$

We also prove an interpretation-relative cofinality theorem for model ramification along definable trees, and a diagonal non-majorization theorem. These results require no assigned proof-theoretic ordinal. The package contains detailed mathematical proofs, new Lean definitions and 84 theorem scripts, exact baseline source copies, and a guarded reference interpreter. The new Lean sources have not been compiler-checked in this environment; 1,830 finite reference-model assertions passed, but kernel certification, axiom closure, and contest build-time compliance remain unverified.

Validation boundary. This is a mathematical research proposal with an unverified Lean formalization, not an already certified contest submission. No Lean compiler was available during preparation, and retrieval attempts did not supply one. The recorded Lean status is `not_run`. There are no intentional proof placeholders in the new sources, but elaboration or proof repairs may still be necessary. Neither contender has been evaluated.

Pinned baseline. All comparisons refer to BusyLean commit `86b360648163f0249eab79539438b6a83dfe8215`.

Target toolchain. Lean 4.34.0; bundled Lean libraries only; no Mathlib.

Contents

1 The result and its intended scope 3

2 What is being extended 3

3 The unchanged object language 5

4 A fixed interface for changing interpretations 6

5 The depth-11 uniform certificate 8

6 Autonomous model ramification 10

7 The complete quantitative proof 11

8 Cofinality of the generated ramification schedule 13

9 Diagonal escape without a halting oracle 14

10 Computability and foundational discipline 14

11 Formalization architecture and actual validation 15

12 Reproduction and integration 17

13 Further research questions 18

14 Conclusion 19

A Complete new computational definition 19

B Correspondence between mathematical statements and sources 22

1 The result and its intended scope

Let $f^{[0]}(x) = x$ and $f^{[k+1]}(x) = f(f^{[k]}(x))$. Throughout this article, square brackets on an exponent denote *function iteration*; they never denote ordinary exponentiation. Define

$$E(x) = 2^x, \qquad \Theta(x) = E^{[x+1]}(x+1). \tag{1}$$

Thus $\Theta(0) = 2$, $\Theta(1) = 16$, and $\Theta(2) = 2^{256}$. For general x, the input determines both the initial exponent and the number of exponentiation steps. In particular, $\Theta(C_6)$ is already much more than an exponential or a fixed-height tower in C_6.

Theorem 1.1 (Main numerical separation)**.** *The construction in Section 6 defines a natural number C_7 using total interpretation and structural recursion. With C_6 the adopted BusyLean number at the pinned revision,*

$$\Theta^{[46]}(C_6) < C_7.$$

Consequently $C_6 < C_7$.

The proof is the specialization at $n = 42$ of the stronger uniform statement

$$\forall n \geq 11, \qquad \Theta^{[n+4]}(\mathrm{F}_6(n)) < \mathrm{C}(M_{\mathrm{final}}, n), \tag{2}$$

where F_6 is the repository's `BusyLean.finalFamily`. The value $46 = 42 + 4$ is an explicitly proved conservative lower bound. It is not an estimate of the entire growth strength of the construction.

The principal advance is structural. The repository's adopted construction runs a fixed reflection mechanism along a program-generated tree. The new construction makes the mechanism itself available to subsequent interpreters, retains complete families of lower mechanisms, and performs *model-valued* recursion along a generated tree. This changes the operations that a short program can call. The numerical certificate in Theorem 1.1 is a transparent way to verify a large gap without assigning ordinals to these operations.

There is no claim of an absolutely largest computable number, of an optimal use of the contest time budget, or of a newly established proof-theoretic ordinal. The direct scalar comparison is with the adopted C_6. The repository's established $C_5 < C_6$ and proposal-P5 comparison yield further corollaries once the new comparison is kernel-checked and linked against the full repository [5]. Exact comparisons with the separate P1–P4 proposal constants are *not* established here. Their heuristic ranking is not a substitute for such proofs.

2 What is being extended

2.1 The current repository, not an earlier proposed contender

The pinned root README and source identify the adopted number as

$$C_6 = \mathrm{F}_6(42), \qquad \mathrm{F}_6(n) = S_{\mathrm{old}}(\mathsf{p}, I_{\mathrm{old}}, n). \tag{3}$$

Here S_{old} is forest reflection over the finite-type W-language, p is P5's proposed number as a family in its input, and I_{old} joins the W-trees denoted by closed programs of that language. These are the actual adopted definitions, not the earlier P5 constant also called `contender_6` in its own namespace [1, 2, 3, 4].

The inherited seed has type $\mathcal{F} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ and is

$$\mathsf{p}(i, n) = \texttt{BigNumber.iterate}\ \texttt{BigNumber.diagonal}\ n\ n.$$

It ignores its first argument. The full definition is retained byte-for-byte in the vendored baseline; it includes P5's seed guard over the old numerical maximum. We do not claim to have eliminated that inherited dependence.

The old reflector is

$$R_{\mathrm{old}}(f, n) = n + 1 + \max\left\{\max_{i \le n} f(i, n),\ \max_{t \in \mathcal{T}_n} \mathrm{eval}_{\mathrm{old},f}(t)\right\}, \tag{4}$$

where $\mathcal{T}_n$ is the repository's finite raw-term enumeration. We use $\max \varnothing = 0$ for numerical maxima throughout. At a leaf, the old forest uses this reflector with the seed; at a node, it uses the family of child stages. Our new forest has the same outer recursive shape, but a strengthened interpretation in its finite maximum.

2.2 Contest obligations and what remains outstanding

The repository rules demand a closed computable natural, a formal strict comparison, no prohibited computational shortcuts or axioms, and clean-build budgets of 15 seconds for definitions and 60 seconds for proofs. They prohibit using the previous named contender as a shortcut and allow only `propext` and `Quot.sound` among axioms. The pinned toolchain is Lean 4.34.0 [1, 10].

The proposed definition never names `BusyLean.contender_6`, `BusyLean.finalFamily`, or `BigNumber.contender_6`. It deliberately reuses their generic constituents: the older reflector, forest operator, index, and P5 seed family. A short program can reconstruct the old final family from those constituents, and this is exactly the comparison witness. Thus the proposal offers syntactic non-use of the old scalar together with a substantial general extension; it does *not* pretend to be semantically independent of the old construction.

The mathematical comparison is proved below. The supplied Lean proof scripts are a concrete formalization attempt, not a certified result. Until the clean build, axiom audit, and dependency audit run successfully, the formal-proof and timing obligations remain open engineering checks for this submission. No elapsed build times are inferred from the size of the sources.

2.3 Reproducible baseline

Three baseline files suffice for the standalone proposed definitions. They were reconstructed from the pinned GitHub contents and checked against their Git blob identifiers:

Vendored path	Bytes
`lean/BigNumber/Definitions.lean`	6,834
`lean/BusyLean/Reflection.lean`	2,841
`lean/BusyLean/Contender6.lean`	2,033

The first file comes from P5's `lean/BigNumber/Definitions.lean`; the other two come from the root BusyLean library. Their exact hashes are recorded in `evidence/vendor_hashes.json`. No upstream compiled binaries or downloaded font files are included.

3 The unchanged object language

3.1 Types, trees, and total interpretation

The finite type codes satisfy

$$A, B ::= \mathsf{nat} \mid A \to B \mid \mathsf{wt}(A).$$

Their denotations are natural numbers, function spaces, and the inductive W-types

$$W(A) = \mathsf{leaf} \mid \mathsf{node}(A \to W(A)).$$

Write $\mathcal{W} = W(\mathbb{N})$. Raw terms have variables, typed abstractions, application, zero, successor, natural recursion, and the leaf, node, and structural recursion operations at all coded W-types. The existing interpreter is total on raw syntax. It returns a type-tagged value and uses the original coercive casts, including defaults for mismatches [4].

This last detail matters. We have not silently replaced the raw interpreter with a different intrinsically typed language. In the numerical maximum, non-numerical results score zero, exactly as in the existing approach. For the particular comparison programs we prove that every application is correctly typed, so the fallback behavior is never used in the successful computation.

A W-node's infinitely many children are represented by a function, not an infinite list that must first be evaluated. Structural recursion on such a node receives recursively defined functions on its children. The closed construction only makes finitely many demands in any terminating natural computation. The governing justification is the accepted structural recursion, not an empirical observation that a few small runs finish.

3.2 Levels, not de Bruijn indices

The repository's variable lookup uses *oldest-first levels*. An abstraction prepends a new binding to the environment list, while level zero continues to refer to the oldest binding. Consequently, adding local binders does not change the level of a designated global interface variable. The new seven-slot environment exploits this convention.

One must distinguish the list order from the level order: the environment list is written newest first. Reversing that list incorrectly would make the comparison program call entirely different values. The semantic proof and reference interpreter explicitly test the intended convention.

3.3 A symbolic rank bound for the enormous enumerations

Let $\mathcal{A}_n$ and $\mathcal{T}_n$ denote the unchanged type and term lists `allTypes n` and `allTerms n`. They are finite recursive lists. At successor depth, the type list contains `nat`, arrows between types in the preceding list, and their W-types. The term list contains variables with levels below the preceding depth, applications of preceding terms, abstractions using preceding type annotations and bodies, and the primitive recursors with preceding annotations.

Define type ranks by

$$\mathrm{rank}_A(\mathsf{nat}) = 1,\ \mathrm{rank}_A(A \to B) = 1 + \max(\mathrm{rank}_A(A), \mathrm{rank}_A(B)),\ \mathrm{rank}_A(\mathsf{wt}(A)) = 1 + \mathrm{rank}_A(A).$$

Term ranks are

$$\begin{aligned} \operatorname{rank}(\mathsf{var}\, i) &= i + 2, \\ \operatorname{rank}(\mathsf{zero}) = \operatorname{rank}(\mathsf{succ}) &= 1, \\ \operatorname{rank}(\mathsf{app}(t, u)) &= 1 + \max(\operatorname{rank}(t), \operatorname{rank}(u)), \\ \operatorname{rank}(\mathsf{lam}(A, B, t)) &= 1 + \max\{\operatorname{rank}_A(A), \operatorname{rank}_A(B), \operatorname{rank}(t)\}, \\ \operatorname{rank}(\mathsf{natrec}(A)) &= 1 + \operatorname{rank}_A(A). \end{aligned}$$

The same one-plus-annotation rule applies to W-leaf and W-node constants; W-recursion has rank one plus the maximum of its two annotation ranks.

Lemma 3.1 (Enumeration inclusion)**.** *If* $\operatorname{rank}_A(A) \le n$*, then* $A \in \mathcal{A}_n$*. If* $\operatorname{rank}(t) \le n$*, then* $t \in \mathcal{T}_n$*.*

Proof. Use structural induction on the type or term, uniformly in the upper bound n. The zero-depth cases are impossible because all ranks are positive. In an application of rank at most $n+1$, both subterms have rank at most n; the induction hypotheses put them in $\mathcal{T}_n$, and the product-list membership lemma puts the application in $\mathcal{T}_{n+1}$. The abstraction case additionally uses the type induction hypotheses. The variable inequality $i + 2 \le n + 1$ gives $i < n$, exactly the membership condition in the variable prefix used for $\mathcal{T}_{n+1}$. Primitive terms follow from the singleton and annotation-mapping parts of the enumerator. These arguments prove the claim at every larger depth without expanding any particular large list. □

This is the route used in `Enumeration.lean`. Checking the proposed comparison need not enumerate $\mathcal{T}_{42}$ or compute its length. The small-depth reference counts in Section 11 are tests of the implementation, not the method of proving the depth-42 membership fact.

4 A fixed interface for changing interpretations

4.1 The model record

A model is a triple of total operations

$$M = (\rho_M, \sigma_M, \tau_M), \quad \begin{cases} \rho_M : \mathbb{N} \to \mathcal{F} \to \mathbb{N} \to \mathbb{N}, \\ \sigma_M : \mathbb{N} \to \mathcal{F} \to \mathcal{W} \to \mathbb{N} \to \mathbb{N}, \\ \tau_M : \mathbb{N} \to \mathcal{W}. \end{cases} \tag{5}$$

The selector argument chooses a lower model. The fields represent its reflection operation, forest recursion, and generated schedule, respectively. A general model need not satisfy any coherence property for the elementary finite-maximum lemmas. The constructors below supply coherent triples for which the selected fields all come from the same lower interpretation.

The interface has a fixed finite type signature. This is important: simply appending a new global constant at every iteration would make certificate addresses and their ranks grow with the iteration count. Here the address stays fixed and a numerical selector chooses the lower interpretation. That selector is itself computed by a term of the object language.

4.2 Seven stable global levels

For a model M, external oracle $f : \mathcal{F}$, and input n, interpretation uses the following oldest-first global levels:

Level	Value	Object-language type
0	External oracle f	$\mathbb{N} \to \mathbb{N} \to \mathbb{N}$
1	Input n	$\mathbb{N}$
2	Inherited seed p	$\mathbb{N} \to \mathbb{N} \to \mathbb{N}$
3	Lower reflectors ρ_M	$\mathbb{N} \to \mathcal{F} \to \mathbb{N} \to \mathbb{N}$
4	Lower forests σ_M	$\mathbb{N} \to \mathcal{F} \to \mathcal{W} \to \mathbb{N} \to \mathbb{N}$
5	Lower schedules τ_M	$\mathbb{N} \to \mathcal{W}$
6	Tower feedback Θ	$\mathbb{N} \to \mathbb{N}$

All displayed types are abbreviations for finite type codes of the unchanged language. The model record itself is a host-level organizing device; the object language sees only these individually typed fields.

Write $\mathrm{eval}_M(f, n, t)$ for the numerical score of the old raw interpreter in this environment. The strengthened reflection operator is

$$R_M(f, n) = n + 1 + \max\left\{\max_{i \le n} f(i, n), \max_{t \in \mathcal{T}_n} \mathrm{eval}_M(f, n, t)\right\}. \tag{6}$$

The forest operator is defined by structural recursion:

$$\begin{aligned} S_M(g, \mathsf{leaf}, n) &= R_M(g, n), \\ S_M(g, \mathsf{node}(a), n) &= R_M\big(i \mapsto (m \mapsto S_M(g, a(i), m)), n\big). \end{aligned} \tag{7}$$

The child is callable at an arbitrary dynamically computed numerical input; we have not replaced the child family by a pointwise maximum.

4.3 Generating schedules at all independent inputs

Let $0_{\mathcal{F}}$ be the zero external oracle. Define

$$\mathrm{treeOf}(M, m, t) = \mathrm{cast}_{\mathcal{W}}\big(\mathrm{interpret}(\mathrm{env}(M, 0_{\mathcal{F}}, m), t)\big).$$

The cast includes the original default behavior when the resulting type is not $\mathcal{W}$. Now set

$$\mathrm{I}(M) = \mathsf{node}\,(d \mapsto \mathsf{node}\,(m \mapsto \mathsf{node}\,(i \mapsto \mathrm{treeOf}(M, m, \mathrm{nthD}(\mathcal{T}_d, i, \mathsf{zero}))))). \tag{8}$$

The three independent indices are syntax depth d, numerical parameter m, and position i in the finite term list. In particular, the construction includes a program evaluated at any fixed input, not merely at an input equal to its enumeration depth or position.

This is a definition by total function application and W-node construction. It does not search over arbitrary programs for a proof of termination or well-foundedness. Every denoted tree is already in the inductive type.

The number associated with a model and an input is

$$\mathrm{C}(M, n) = S_M(\mathsf{p}, \mathrm{I}(M), n). \tag{9}$$

The final contender will be one instance of this uniform construction.

5 The depth-11 uniform certificate

5.1 The certificate program

For a selector term s, write the left-associated application

$$\mathrm{call}(s) = \big(((v_4\ s)\ v_2)\ (v_5\ s)\big)\ v_1, \qquad A(s) = v_6\ \mathrm{call}(s), \tag{10}$$

where v_j denotes the variable at level j. Informally, the program selects a lower forest, gives it the inherited seed and matching lower schedule, evaluates at the current input, and applies tower feedback to the result.

Lemma 5.1 (Exact rank)**.** *For every selector s,*

$$\mathrm{rank}(A(s)) = \max\{11, \mathrm{rank}(s) + 5\}.$$

In particular, $\mathrm{rank}(A(\mathsf{zero})) = \mathrm{rank}(A(v_1)) = 11$.

Proof. Let $r = \mathrm{rank}(s)$. The variable ranks of v_4, v_5, v_6, v_2, v_1 are $6, 7, 8, 4, 3$. The first application $v_4\ s$ has rank $\max(7, r+1)$; applying to v_2 gives $\max(8, r+2)$. The other branch $v_5\ s$ has rank $\max(8, r+1)$, so the next application has rank $\max(9, r+3)$. Applying to v_1 gives $\max(10, r+4)$, and the final application of v_6 gives $\max(11, r+5)$. Finally, zero has rank 1 and v_1 has rank 3. □

Thus a single small syntax tree certifies a comparison for all models and all $n \geq 11$. Its rank does not grow with the complexity of the model stored in the environment.

5.2 Why casts must be justified

A correctly tagged application still uses the repository's `castValue` function. At W-types, a self-cast traverses the tree with an identity branch map, so it should not be dismissed as a purely definitional identity.

Lemma 5.2 (Self-casts)**.** *For every finite type code A and every $x \in [\![A]\!]$,* $\mathrm{castValue}(A, x) = x$.

Proof. First prove that the Boolean type equality test recognizes $A = A$, by type induction. For every $w \in W(B)$, prove $\mathrm{wmap}(\mathrm{id}, w) = w$ by W-induction. The leaf case is immediate. At a node, the induction hypotheses identify the child functions pointwise; function extensionality identifies the functions themselves, and congruence identifies the nodes.

Now induct on A to show that the pair of casts from A to itself is $(\mathrm{id}, \mathrm{id})$. The natural case is immediate. In an arrow case, the recursively supplied domain and codomain casts are identities, so both composite functions are identities. In a W-case, apply the identity branch-map result. The desired statement is the first component of this pair identity applied to x. □

The proposed Lean proof uses ordinary function extensionality, which is within the repository's allowed axiom framework. Its actual transitive axiom set is still a pending compiler audit. No equality of different tree notations or ordinal representations is needed.

Lemma 5.3 (Certificate semantics)**.** *Suppose s evaluates to the natural k in the model environment. Then*

$$\mathrm{eval}_M(f, n, A(s)) = \Theta(\sigma_M(k, \mathsf{p}, \tau_M(k), n)). \tag{11}$$

For $s = \mathsf{zero}$ or $s = v_1$, this value is independent of the external oracle f, with $k = 0$ or $k = n$, respectively.

Proof. The level lookup gives exactly the values listed in Section 4. Every application in (10) matches its declared domain and codomain. Lemma 5.2 therefore removes the inserted self-casts. Applying the selected forest to the seed, the selected schedule, and the input yields $\sigma_M(k, \mathsf{p}, \tau_M(k), n)$. Level 6 then applies Θ. Neither zero nor the input variable consults level 0, and no other part of this certificate consults that level. □

5.3 Lifting a witness through every forest

Lemma 5.4 (Strict reflection bounds)**.** *For every model M, family f, and input n:*

$$\begin{aligned} n &< R_M(f, n), \\ i \le n &\Longrightarrow f(i, n) < R_M(f, n), \\ t \in \mathcal{T}_n &\Longrightarrow \mathrm{eval}_M(f, n, t) < R_M(f, n). \end{aligned}$$

Proof. Every indicated value, except the already separately handled n, is at most the maximum in (6). The positive summand $n + 1$ makes the inequality strict. Also $R_M(f, n) \ge n + 1$ because its maximum is nonnegative. The prefix and list maximum bounds are elementary inductions on the natural prefix length and list membership. □

Theorem 5.5 (Oracle-uniform witness lifting)**.** *Fix M, n, t with $t \in \mathcal{T}_n$. If a natural b satisfies*

$$\forall f : \mathcal{F}, \quad \mathrm{eval}_M(f, n, t) = b,$$

then for every seed g and every tree $a \in \mathcal{W}$,

$$b < S_M(g, a, n).$$

Proof. Split into the leaf and node cases of a. At a leaf, use Lemma 5.4 with $f = g$. At a node, use the same lemma with $f(i, m) = S_M(g, a(i), m)$. The hypothesis explicitly quantifies over this family, so the value is still b. No induction hypothesis or comparison between the different oracles is required. □

Combining the preceding results yields, for every M and $n \ge 11$,

$$\Theta(\sigma_M(0, \mathsf{p}, \tau_M(0), n)) < \mathrm{C}(M, n), \tag{12}$$

$$\Theta(\sigma_M(n, \mathsf{p}, \tau_M(n), n)) < \mathrm{C}(M, n). \tag{13}$$

These two inequalities drive the entire numerical proof.

Remark 5.6 (No oracle-monotonicity shortcut)**.** An object program can test whether an oracle result is zero; thus increasing an oracle pointwise can decrease that program's output. This observation alone does *not* prove that the maximum defining R_M is nonmonotone, since a maximum can behave differently from an individual member. It does show why pointwise comparison of oracle families cannot simply be pushed through an arbitrary program. Our proofs make no such inference about either programs or reflectors. They use a fixed oracle-independent witness instead.

6 Autonomous model ramification

6.1 One step and all finite steps

Define the successor model by exposing the current model's operations:

$$\begin{aligned} \rho_{\mathrm{Next}(M)}(k) &= R_M, \\ \sigma_{\mathrm{Next}(M)}(k) &= S_M, \\ \tau_{\mathrm{Next}(M)}(k) &= \mathrm{I}(M). \end{aligned} \tag{14}$$

The selector is ignored at this one-step interface. Let

$$M^{(0)} = M, \qquad M^{(k+1)} = \mathrm{Next}(M^{(k)}).$$

The closure under all finite steps, denoted $\Omega(M)$, has fields

$$\rho_{\Omega(M)}(k) = R_{M^{(k)}}, \quad \sigma_{\Omega(M)}(k) = S_{M^{(k)}}, \quad \tau_{\Omega(M)}(k) = \mathrm{I}(M^{(k)}). \tag{15}$$

The notation Ω is a convenient name for this operator. It is *not* an asserted ordinal notation or an identification with a set-theoretic ordinal. The defining recursion for $M^{(k)}$ is ordinary natural recursion.

A program can calculate a selector k much larger than its own input or syntax depth. The specific lower bound will only need selectors zero and n, but the full operator retains more computational capability.

6.2 Retaining a base and every child

For a base model B and a family $i \mapsto M_i$, let $J = \mathrm{Join}(B, (M_i)_i)$ expose the operations of B at selector zero and of M_i at selector $i+1$:

$$\begin{array}{lll} \rho_J(0) = R_B, & \sigma_J(0) = S_B, & \tau_J(0) = \mathrm{I}(B), \\ \rho_J(i+1) = R_{M_i}, & \sigma_J(i+1) = S_{M_i}, & \tau_J(i+1) = \mathrm{I}(M_i). \end{array} \tag{16}$$

The base is retained even when there are children. This explicit zero slot is what makes the uniform base lower bound independent of which schedule is used. Each child is retained as a full interpreter interface, rather than as one numerical output.

Define ramification by structural recursion on $a \in \mathcal{W}$:

$$\begin{aligned} \mathrm{Ram}(B, \mathsf{leaf}) &= \Omega(B), \\ \mathrm{Ram}(B, \mathsf{node}(a)) &= \Omega\big(\mathrm{Join}(B, (\mathrm{Ram}(B, a(i)))_i)\big)\,. \end{aligned} \tag{17}$$

Thus every node collects all the recursively constructed child models and closes the resulting interface under all finite further steps. This recursion has values in the fixed host type `Model`. It does not quantify over all host-level definitions and does not require an internal truth predicate.

6.3 The closed final model

The initial model M_0 exposes the existing components:

$$\rho_{M_0}(k) = R_{\mathrm{old}}, \qquad \sigma_{M_0}(k) = S_{\mathrm{old}}, \qquad \tau_{M_0}(k) = I_{\mathrm{old}}. \tag{18}$$

The closed schedule and final model are

$$\begin{aligned} M_\omega &= \Omega(M_0), \\ a_* &= \mathrm{I}(M_\omega), \\ M_* &= \mathrm{Ram}(M_\omega, a_*), \\ M_{\mathrm{final}} &= \Omega(M_*), \\ C_7 &= \mathrm{C}(M_{\mathrm{final}}, 42). \end{aligned} \tag{19}$$

The defining dependency graph has the forward order

$$M_0 \longrightarrow M_\omega \longrightarrow a_* \longrightarrow M_* \longrightarrow M_{\mathrm{final}} \longrightarrow C_7.$$

The schedule is generated by an *already defined lower* model. The ramified model does not occur in the definition of its own schedule. The term "autonomous" refers to the language-generated choice of schedule, not to a circular equation for a universal self-interpreter.

7 The complete quantitative proof

7.1 Elementary facts about tower feedback

Lemma 7.1. *The function Θ is monotone on $\mathbb{N}$, and $x < \Theta(x)$ for every x. Every iterate $\Theta^{[k]}$ is monotone and inflationary.*

Proof. First, $2^x > x$ for every natural x. At zero this is $1 > 0$; the successor step follows from $2^{x+1} = 2 \cdot 2^x$ and $2^x \geq x + 1$. The function $E(x) = 2^x$ is monotone. Its iterates are therefore monotone in the starting value by induction, and inflationary because each individual step is inflationary. They are also monotone in the iteration count.

If $x \leq y$, apply these two monotonicity facts to the initial values $x + 1 \leq y + 1$ and counts $x + 1 \leq y + 1$ to obtain $\Theta(x) \leq \Theta(y)$. Furthermore, the initial value $x + 1$ is preserved or increased by its $x + 1$ iterations, so $x < x + 1 \leq \Theta(x)$. Induction gives monotonicity and inflationarity of every Θ iterate. □

For clarity, none of the comparison arguments below requires strict monotonicity of Θ. Ordinary monotonicity plus the strict inequalities from reflection suffices.

7.2 Successor, finite, and omega bounds

Proposition 7.2 (Model amplification)**.** *For every model M and $n \geq 11$,*

$$\Theta(\mathrm{C}(M, n)) < \mathrm{C}(\mathrm{Next}(M), n), \tag{20}$$

$$\Theta^{[k]}(\mathrm{C}(M, n)) \leq \mathrm{C}(M^{(k)}, n) \qquad (k \in \mathbb{N}), \tag{21}$$

$$\Theta(\mathrm{C}(M^{(n)}, n)) < \mathrm{C}(\Omega(M), n), \tag{22}$$

$$\Theta^{[n+1]}(\mathrm{C}(M, n)) < \mathrm{C}(\Omega(M), n), \tag{23}$$

$$\Theta(\mathrm{C}(M, n)) < \mathrm{C}(\Omega(M), n). \tag{24}$$

Proof. Use the zero-selector certificate (12) in the model $\mathrm{Next}(M)$. By its field definitions, the selected forest and schedule are S_M and $\mathrm{I}(M)$, so this is exactly (20).

For (21), induct on k. The base case is equality. Given the bound at k, monotonicity of Θ gives

$$\Theta^{[k+1]}(\mathrm{C}(M,n)) \le \Theta(\mathrm{C}(M^{(k)},n)) < \mathrm{C}(M^{(k+1)},n).$$

Dropping strictness proves the desired successor inequality.

Use the input-selector certificate (13) in $\Omega(M)$. Its selector n exposes $M^{(n)}$, giving (22). Applying Θ to (21) with $k = n$ and composing proves (23). Finally, using the zero-selector certificate instead exposes $M^{(0)} = M$ and proves (24) directly. □

Proposition 7.3 (Preservation through joins and ramification)**.** *For every base B, every child family (M_i), every tree a, and $n \ge 11$,*

$$\Theta(\mathrm{C}(B,n)) < \mathrm{C}(\mathrm{Join}(B,(M_i)),n), \tag{25}$$

$$\Theta(\mathrm{C}(B,n)) < \mathrm{C}(\mathrm{Ram}(B,a),n). \tag{26}$$

Proof. The zero slot of the join contains precisely S_B and $\mathrm{I}(B)$. Thus (25) is another instance of the zero-selector certificate. For (26), split into the tree's two constructors. At a leaf, use (24). At a node, write $J = \mathrm{Join}(B,(\mathrm{Ram}(B,a(i)))_i)$. Then

$$\Theta(\mathrm{C}(B,n)) < \mathrm{C}(J,n) < \Theta(\mathrm{C}(J,n)) < \mathrm{C}(\Omega(J),n).$$

The first inequality is the join bound, the middle one is Lemma 7.1, and the last is the omega base bound. The rightmost value is the required ramification value. No inductive comparison of the children is needed for this particular lower bound. □

7.3 From the old family to the final model

By the fields of M_0, the zero-selector certificate reconstructs the old family exactly. For every $n \ge 11$,

$$\Theta(\mathrm{F}_6(n)) < \mathrm{C}(M_0,n). \tag{27}$$

This is an equality-of-computation comparison, not an ordinal estimate.

Theorem 7.4 (Uniform separation)**.** *For every natural $n \ge 11$,*

$$\Theta^{[n+2]}(\mathrm{F}_6(n)) < \mathrm{C}(M_\omega,n), \tag{28}$$

$$\Theta^{[n+3]}(\mathrm{F}_6(n)) < \mathrm{C}(M_*,n), \tag{29}$$

$$\Theta^{[n+4]}(\mathrm{F}_6(n)) < \mathrm{C}(M_{\mathrm{final}},n). \tag{30}$$

Proof. Apply the monotone function $\Theta^{[n+1]}$ to the non-strict consequence of (27), then use (23):

$$\Theta^{[n+2]}(\mathrm{F}_6(n)) \le \Theta^{[n+1]}(\mathrm{C}(M_0,n)) < \mathrm{C}(\Omega(M_0),n).$$

This proves (28). Apply Θ to its non-strict consequence and then use (26) with base M_ω and tree a_*. The result is (29). Apply Θ once more and use (24) with $M = M_*$ to get (30). □

Theorem 1.1 now follows by taking $n = 42$, using $C_6 = \mathrm{F}_6(42)$ and the definition of C_7. Inflationarity gives $C_6 \le \Theta^{[46]}(C_6)$, so the ordinary strict comparison follows too.

The proof has deliberately not used most of the strength of the final omega closure or the structure of a_*. It establishes a readily inspectable numerical margin first. The next section explains a structural guarantee that would be lost if the schedule were replaced by an arbitrary single leaf.

8 Cofinality of the generated ramification schedule

8.1 Selecting a particular child

Let $\overline{k}$ be the unary numeral obtained from zero by k applications of successor. Then $\mathrm{rank}(\overline{k}) = k+1$, and

$$\mathrm{rank}(A(\overline{k})) = \max\{11, k+6\}. \tag{31}$$

The use of unary numerals is a proof convenience, not a claim that it gives the most compact selectors. The full language can compute much larger selectors using shorter programs.

Lemma 8.1 (Visible child amplification)**.** *Let $a : \mathbb{N} \to \mathcal{W}$. If $n \geq 11$ and $i + 7 \leq n$, then*

$$\Theta\big(\mathrm{C}(\mathrm{Ram}(B, a(i)), n)\big) < \mathrm{C}(\mathrm{Ram}(B, \mathsf{node}(a)), n).$$

In particular the same inequality holds with Θ omitted from the left.

Proof. Set $M_i = \mathrm{Ram}(B, a(i))$ and $J = \mathrm{Join}(B, (M_i))$. The term $A(\overline{i+1})$ has rank $\max(11, i+7) \leq n$, independently of the external oracle. The selected fields of J are S_{M_i} and $\mathrm{I}(M_i)$. The uniform witness lemma gives

$$\Theta(\mathrm{C}(M_i, n)) < \mathrm{C}(J, n).$$

The omega base bound gives $\mathrm{C}(J, n) < \Theta(\mathrm{C}(J, n)) < \mathrm{C}(\Omega(J), n)$. The last model is exactly $\mathrm{Ram}(B, \mathsf{node}(a))$. Inflationarity of Θ gives the unamplified consequence. □

8.2 Every fixed definable schedule is eventually dominated

Theorem 8.2 (Interpretation-relative cofinality)**.** *For every base model B, model M, raw term t, and natural parameter m, there is a natural N such that for every $n \geq N$,*

$$\mathrm{C}\big(\mathrm{Ram}(B, \mathrm{treeOf}(M, m, t)), n\big) < \mathrm{C}\big(\mathrm{Ram}(B, \mathrm{I}(M)), n\big). \tag{32}$$

Proof. Let $d = \mathrm{rank}(t)$. Lemma 3.1 gives $t \in \mathcal{T}_d$. A simple induction on list membership supplies some position i for which $\mathrm{nthD}(\mathcal{T}_d, i, \mathsf{zero}) = t$. Choose

$$N = \max\{11, d+7, m+7, i+7\}. \tag{33}$$

For $n \geq N$, apply the unamplified conclusion of Lemma 8.1 three times: first to position i, then to parameter m, and finally to depth d. The three node layers are exactly those in (8). Substituting the list-position equality at the innermost leaf gives (32). □

For the final construction, take $B = M = M_\omega$. The intermediate model M_* eventually dominates the model ramification along every fixed tree that the initially omega-strengthened interpretation produces, at every fixed numerical parameter. The final omega closure further dominates this intermediate model at all sufficiently large inputs.

Several scope restrictions are essential. The threshold can be large because the position in a finite enumeration can be large. The theorem therefore does not say that every arbitrarily complicated schedule is already dominated at input 42. It is relative to a specified model's denotation of raw terms, not to the class of all Lean-definable trees or all computable well-orders. It is a numerical comparison of the resulting ramification constructions, not an assertion that the index has been assigned a particular ordinal rank.

9 Diagonal escape without a halting oracle

Theorem 9.1 (No eventual self-majorant)**.** *Fix a model M, a raw term t, and any input-dependent family of external oracles $n \mapsto f_n$. There is no N such that*

$$\forall n \geq N, \qquad R_M(f_n, n) \leq \mathrm{eval}_M(f_n, n, t).$$

Proof. Suppose such an N exists and let $n = \max\{N, \mathrm{rank}(t)\}$. By Lemma 3.1, $t \in \mathcal{T}_n$. The strict program bound of Lemma 5.4 gives

$$\mathrm{eval}_M(f_n, n, t) < R_M(f_n, n) \leq \mathrm{eval}_M(f_n, n, t),$$

a contradiction. □

This diagonal theorem concerns a *fixed* interpretation with its fixed interface. The successor construction can denote computations associated with the lower interpretation because it changes that interface. Consequently, there is no contradiction between the diagonal theorem and the successful short certificate that calls the lower forest in the next model.

Nor is there an appeal to a Busy Beaver maximum. The finite lists contain terms of an independently total interpreted language. We neither filter all Turing programs by whether they halt nor assign values using a noncomputable halting predicate. The diagonal quantifiers range over the syntax of the specified language, not over every possible terminating host-language program.

10 Computability and foundational discipline

10.1 Why the closed number is computable in principle

The proposed computation uses the following forms of recursion. The existing interpreter recurses structurally on raw syntax, with natural and W-recursion provided at the denoted types. Prefix maxima and finite term lists use natural or list recursion. The new forest operator recurses structurally on its W-tree. The finite model tower uses natural recursion, and model ramification uses W-tree recursion. All later expressions in (19) are finite compositions of these operations.

This is a constructive recipe, not a classical existence definition of a natural. Once the definitions are accepted by the pinned Lean checker, their termination is supported by the same inductive-recursion discipline used by the baseline [8, 9]. Independently of compiler acceptance, the displayed mathematical recursions give the total construction and its proofs; the exact Lean source's acceptance is the separate validation question recorded in this package.

For a generic argument M, the mathematical model record permits arbitrary total functions, which need not be computable. No claim is made that $\mathrm{C}(M, n)$ is uniformly computable given a noncomputable external model. The computability assertion is about the *closed* M_0 and the explicit constructors applied to it. Every model field in that closed dependency graph has an algorithmic definition; no noncomputable data are supplied at a leaf.

There is no practical route to writing down the final decimal expansion. Computability in principle is not a resource-feasibility claim. Even the stated lower bound is beyond physical evaluation, and executing the finite maximum at depth 42 would itself be inappropriate. The build and proof strategy is to check symbolic definitions and inequalities while never normalizing either contender.

10.2 No hidden universes or paradoxical self-evaluation

The new record has a fixed collection of ordinary higher-order function fields. Iterating its construction does not require extending the object type syntax with a universe of all models, nor with a type of all host programs. The same finite type codes interpret each field at every stage. The host language defines the family of interpretations using its own structural recursion.

Similarly, $\mathrm{I}(M)$ is not defined as a join of all trees expressible using $\mathrm{I}(M)$ as a new primitive. It is defined using the operations *already stored in* M. Only after that index has been defined can `next M` expose it. This sequencing is exactly what keeps the diagonal argument and the construction compatible.

The use of extensional equality in the cast proof is explicit. There is no `Classical.choice`, admitted theorem, custom axiom, or unsafe evaluation mechanism intentionally present in the new source. This source-level design statement is weaker than a completed transitive axiom audit. The latter must be obtained from the compiled environment, not from a grep result.

10.3 What has and has not been measured by ordinals

The upstream review describes rough ordinal calibrations for the language and its proposals, and labels them informal [6]. We do not use those calibrations as premises. A proof-theoretic ordinal of a theory, an ordinal notation system representing tree schedules, and a finite numerical comparison at input 42 are different mathematical objects. Identifying one with another requires a precisely stated translation and quantitative control of fundamental sequences or coding costs.

The present contribution avoids this gap. It gives an exact program witness, an exact rank cutoff, exact iteration inequalities, and an explicit eventual cofinality threshold. It is meaningful even without determining which ordinal-analysis framework best describes the overall model hierarchy.

11 Formalization architecture and actual validation

11.1 Source modules

The new library consists of 823 lines across eight modules, with 32 `def` declarations and 84 `theorem` declarations according to the included lexical audit. The declaration counts exclude structures and abbreviations and are not a proof-complexity measure.

Module	Mathematical role
`Definitions.lean`	Fixed model interface, new reflection and forest, generated index, model constructors, closed contender.
`Arithmetic.lean`	Inflationarity and monotonicity of exponential feedback and its iterations.
`Enumeration.lean`	Type/term rank inclusion and the depth-11 certificates, without expanding large enumerations.
`Semantics.lean`	Self-cast identity, exact typed application, certificate evaluation.
`Reflection.lean`	Finite maximum inequalities, oracle-uniform witness lifting, diagonal non-majorization.
`Comparison.lean`	Successor/omega/ramification bounds and the 46-fold comparison with the pinned contender.
`Selectors.lean`	Unary selectors and the explicit child visibility cost.
`Cofinality.lean`	Every fixed definable schedule is eventually dominated by the generated ramification schedule.

The actual proposed scalar theorem names are

```
BusyLean.contender_6_tower_lt_contender_7
BusyLean.contender_6_lt_contender_7
```

The strongest uniform numerical theorem is `BusyLean7.final_bound`; the structural theorem is `BusyLean7.ramify_index_eventually_dominates`. The package's root `BusyLean7.lean` imports both comparison and cofinality.

11.2 Independent finite tests

The Python reference implementation mirrors the raw syntax constructors, finite type denotations, casting convention, oldest-first lookup, and small finite enumerations. It has explicit safeguards against accidentally attempting large computations. The test run recorded in `evidence/python_tests.json` passed 1,830 assertions.

Depth	0	1	2	3	4
Number of raw terms	0	2	13	308	147,129
Mock-model numerical maximum	0	0	1	3	5

The second row of numbers concerns a deliberately small *mock model*; it is not a sequence of actual values of C_6, C_7, or the proposed self-ramified model. The tests also include 976 checks of the comparison certificate across different numerical inputs, external oracles, and selectors, as well as 48 small reflector checks. The tested tower values are 2, 16, and 2^{256}. Evaluation beyond the guarded tiny cases is refused.

These checks are useful for finding level-order errors, wrong application nesting, incorrect syntax-rank offsets, and mistakes in the fallback cast behavior. They cannot establish the universal claims of the article and cannot establish that Lean accepts a proof term. In particular, a passing independent interpreter cannot serve as a substitute for the Lean kernel.

11.3 The audit ledger

Check	Recorded outcome
Pinned baseline Git blob identities	All three matched exactly.
Finite Python reference tests	Passed; 1,830 assertions.
Lexical scan of new computational sources	No prohibited tokens found; no direct reference to the forbidden named winners.
New Lean definition elaboration	Not run; compiler unavailable.
New Lean proof checking	Not run; compiler unavailable.
Transitive axiom/dependency audit	Supplied, but not run.
15-second / 60-second clean-build budgets	Unverified.
Evaluation of either contender	Not attempted.

A lexical scan is not a parser, a termination checker, or a dependency-closure computation. The proposed Lean audit separately traverses constants in both types and values, prints the axioms of submitted roots, and tests forbidden constant reachability. Its design follows the pinned repository's audit pattern [7]; its execution remains outstanding.

No `sorry` or custom axiom is inserted to make an apparent build succeed. If the first real compiler run reports an error, that should be repaired and recorded honestly rather than suppressed. The source is modular precisely so that enumeration, semantic casts, numerical comparisons, and cofinality can be checked in that order without evaluating a huge natural.

12 Reproduction and integration

The ZIP is standalone for the proposed new definition and direct comparison: the three necessary old definition files are vendored. It does not require fetching the full repository or Mathlib merely to inspect or attempt the new build. The toolchain itself must be installed separately.

From the package root, the available finite checks are

```
python3 python/test_reference.py
python3 scripts/source_audit.py
```

With Lean 4.34.0 and Lake installed, run

```
python3 scripts/check.py
```

The script checks the toolchain, removes the package's Lean build directory, builds the definition phase and proof phase separately, runs safe smoke tests, and runs the axiom/dependency audit. Missing Lean yields a non-success result and an explicit `not_run` receipt, not a fabricated pass. The script also records whether measured definition and proof times meet the repository budgets. A plain attempted library build is

```
cd lean
lake build BusyLean7
```

The optional `integration/Transitive.lean` imports the full pinned repository's existing `BusyLean/Comparison.lean` and composes its two strict inequalities with the proposed $C_6 < C_7$ theorem. That optional file is outside the standalone build because the old comparison has additional port-agreement dependencies that are not needed for the new direct bound. It provides the intended statements

```
BusyLean.contender_5_lt_contender_7
BusyLean.p5_lt_contender_7
```

without modifying or redefining the old contender constants.

For integration into the original repository, copy only the new namespace modules and the desired tests, retaining the pinned baseline references for review. The old files in this ZIP are source witnesses, not proposed changes to their upstream contents. See `docs/INTEGRATION.md` for the concrete layout and validation order.

Build this article with

```
bash scripts/build_pdf.sh
```

The script runs three `pdflatex` passes and writes build logs to the evidence directory. The delivered PDF was compiled from the accompanying source; its validation receipt is separate from the Lean receipt.

13 Further research questions

13.1 Complete kernel certification before extending the hierarchy

The immediate priority is a Lean 4.34.0 build of all eight modules, followed by the transitive axiom/dependency audit and the clean timing measurement. The generic mathematical lemmas have intentionally been separated from scalar specialization to reduce accidental normalization. Any required elaboration repairs should preserve the defining equations and the exact rank-11 certificate, with a new validation receipt explaining the changes. Only after that pass should the proposal be described as a certified contest entry.

13.2 Certify the other proposal constants by translations

An exact $C_7 > P_j$ statement for each of P1–P4 requires a concrete comparison, not reliance on the repository's heuristic ranking. One route is to encode their primitive tree operations and iteration functionals as fixed programs in a sufficiently low model, prove semantic agreement, and track syntax rank and parameter overhead. Another is a direct uniform interpretation of each proposal's definition into the current forest interface. The finite input 42 makes overhead accounting essential; eventual domination by itself does not settle the scalar inequality.

13.3 Reduce selection overhead without changing semantics

The threshold $i + 7 \leq n$ in the cofinality proof comes from a unary numeral. Replacing it with a certified binary or arithmetic construction should make many branches visible at much smaller input. The correct theorem would state the cost in terms of the length of a verified selector program, not in terms of the child index itself. A general oracle-uniform selector lemma is already present conceptually; extending its library is a natural next step. The three independent schedule indices should be retained during such a compression.

13.4 Use more of the final model in the numerical lower bound

The final proof only uses the zero slot of the outermost omega closure and a base-preservation bound through the generated schedule. Stronger explicit bounds should exploit the input selector there, and then use identified branches of a_* with small certified positions. The challenge is not merely obtaining a larger expression, but doing so with a transparent comparison theorem that avoids implicit monotonicity assumptions about interpreters.

13.5 Develop a precise ordinal analysis separately

A systematic calibration could define a notation system for the model constructors, prove a simulation theorem relating that system to the W-tree semantics, and derive upper and lower bounds for numerical evaluations. Each bridge should specify its coding costs and parameter shifts. It would be inappropriate to identify $\Omega(M)$ with a familiar ordinal operation solely because both involve a countable family. The exact results in this article can serve as constraints and test cases for such an analysis.

13.6 Generalize the interface and retain quantitative witnesses

The fixed-signature method applies whenever a total language can expose families of lower evaluators and their recursion schedules. Higher-type selectors, richer strictly positive data types, or multiple interacting schedule families could produce stronger variants. The key design target is an invariant short witness whose semantic value is independent of the current external oracle. Without such a witness, plausible structural strengthening can become difficult to compare at a fixed natural input.

14 Conclusion

The new number is obtained from a hierarchy of interpreters rather than from an unexplained enormous numeral or a halting oracle. A fixed interface makes lower reflection, forest recursion, and language-generated schedules available to the next interpretation. Natural iteration and structural model-valued W-recursion organize those interpretations without circular self-reference.

A single depth-11 certificate proves a large, explicit, uniform gap. The result at the repository input is $\Theta^{[46]}(C_6) < C_7$. The generated schedule additionally satisfies an exact eventual domination theorem for ramification along every fixed tree definable in the lower model. These are proved mathematical statements about the displayed construction, not heuristic ordinal rankings.

The remaining boundary is computational verification of the submitted Lean text. The package states that boundary prominently, provides reproducible checking and audit scripts, and records only checks actually performed. This keeps the substantial mathematical proposal separate from the certification work that still has to be completed.

A Complete new computational definition

The following is the actual definition module included in the package. It imports the pinned current definition to reuse the generic old components; its new contender body does not reference the old contender or final unary family. Proofs and the transitive dependency audit are in separate files.

```
import BusyLean.Contender6

/-!
# Autonomous model ramification: definitions only

Pinned baseline: BusyLean 86b360648163f0249eab79539438b6a83dfe8215.
This file has not been compiler-checked in the authoring environment.
No evaluation of the final contender is requested or appropriate.
-/
namespace BusyLean7

open BigNumber (Ty Term Value Env)
abbrev Tree := BigNumber.WTree Nat
abbrev Family := BusyLean.W.Family

/-- Ordinary iteration, with a structurally decreasing natural count. -/
def iter {A : Type} (f : A → A) : Nat → A → A
  | 0, x => x
  | k + 1, x => f (iter f k x)

/-- Exponential tower with height AND starting value supplied by the input. -/
def theta (x : Nat) : Nat := iter BigNumber.pow2 (x + 1) (x + 1)

/-- The fixed interface never grows when the interpretation is strengthened. -/
structure Model where
  lowerReflect : Nat → Family → Nat → Nat
  lowerStage : Nat → Family → Tree → Nat → Nat
  lowerTree : Nat → Tree

def numFnTy : Ty := .arr .nat .nat
def familyTy : Ty := .arr .nat numFnTy
def treeTy : Ty := .wt .nat
def reflectTy : Ty := .arr familyTy numFnTy
def stageTy : Ty := .arr familyTy (.arr treeTy numFnTy)

/-- Oldest-first levels: 0=oracle, 1=input, 2=seed, 3=reflector family,
4=forest family, 5=tree family, 6=tower feedback. This order is invariant. -/
def modelEnv (M : Model) (fs : Family) (n : Nat) : Env :=
  [⟨numFnTy, theta⟩,
   ⟨.arr .nat treeTy, M.lowerTree⟩,
   ⟨.arr .nat stageTy, M.lowerStage⟩,
   ⟨.arr .nat reflectTy, M.lowerReflect⟩,
   ⟨familyTy, BusyLean.p5Family⟩,
   ⟨.nat, n⟩,
   ⟨familyTy, fs⟩]

def score (v : Value) : Nat :=
  match v with
  | ⟨.nat, n⟩ => n
  | ⟨.arr _ _, _⟩ => 0
  | ⟨.wt _, _⟩ => 0

def eval (M : Model) (fs : Family) (n : Nat) (t : Term) : Nat :=
  score (BigNumber.interpret (modelEnv M fs n) t)

/-- A finite, oracle-relative reflection operation. -/
def reflect (M : Model) (fs : Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (BusyLean.W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum (eval M fs n) (BigNumber.allTerms n))

/-- Forest reflection with the strengthened evaluator. -/
def stage (M : Model) (seed : Family) : Tree → Nat → Nat
  | .leaf => reflect M seed
  | .node f => reflect M (fun i => stage M seed (f i))
termination_by structural a => a

def zeroFamily : Family := fun _ _ => 0
```

```
def treeOf (M : Model) (input : Nat) (t : Term) : Tree :=
  BigNumber.castValue treeTy
    (BigNumber.interpret (modelEnv M zeroFamily input) t).2

/-- Three independent indices: syntax depth, numerical parameter, program
position. No search for a well-foundedness certificate is involved. -/
def modelIndex (M : Model) : Tree :=
  .node fun depth => .node fun input => .node fun position =>
    treeOf M input
      (BusyLean.nthD (BigNumber.allTerms depth) position .zero)

/-- Internalize the current evaluator's three operations, separately. -/
def next (M : Model) : Model where
  lowerReflect := fun _ => reflect M
  lowerStage := fun _ => stage M
  lowerTree := fun _ => modelIndex M

/-- A finite ramification tower. -/
def finite (M : Model) : Nat → Model
  | 0 => M
  | k + 1 => next (finite M k)

/-- A program may select its ramification level by an arbitrary computation. -/
def omegaLift (M : Model) : Model where
  lowerReflect := fun k => reflect (finite M k)
  lowerStage := fun k => stage (finite M k)
  lowerTree := fun k => modelIndex (finite M k)

/-- Slot 0 preserves the base; slot i+1 exposes the complete child model i. -/
def join (base : Model) (children : Nat → Model) : Model where
  lowerReflect := fun k => match k with
    | 0 => reflect base
    | i + 1 => reflect (children i)
  lowerStage := fun k => match k with
    | 0 => stage base
    | i + 1 => stage (children i)
  lowerTree := fun k => match k with
    | 0 => modelIndex base
    | i + 1 => modelIndex (children i)

/-- Model-valued recursion along a well-founded schedule. Each node closes
under all finite further ramifications, while retaining every child. -/
def ramify (base : Model) : Tree → Model
  | .leaf => omegaLift base
  | .node f => omegaLift (join base (fun i => ramify base (f i)))
termination_by structural a => a

/-- Existing COMPONENTS, not the old number or its final unary family. -/
def baseModel : Model where
  lowerReflect := fun _ => BusyLean.W.reflect
  lowerStage := fun _ => BusyLean.W.stage
  lowerTree := fun _ => BusyLean.index

def initialOmega : Model := omegaLift baseModel
/-- Generated by the already strengthened language, at all natural inputs. -/
def selfSchedule : Tree := modelIndex initialOmega
def ramifiedModel : Model := ramify initialOmega selfSchedule
def finalModel : Model := omegaLift ramifiedModel

def value (M : Model) (n : Nat) : Nat :=
  stage M BusyLean.p5Family (modelIndex M) n

/-- Certificate syntax: call the selected forest on the selected index. -/
def callAt (selector : Term) : Term :=
  .app
    (.app
      (.app (.app (.var 4) selector) (.var 2))
```

```
      (.app (.var 5) selector))
    (.var 1)

/-- One uniform certificate adds tower-height feedback to that computation. -/
def amplifiedCall (selector : Term) : Term :=
  .app (.var 6) (callAt selector)

end BusyLean7

namespace BusyLean
/-- Proposed Contender 7. NEVER evaluate this definition. -/
def contender_7 : Nat := BusyLean7.value BusyLean7.finalModel 42
end BusyLean
```

B Correspondence between mathematical statements and sources

Statement	Proposed Lean declaration
Enumeration inclusion	`BusyLean7.term_mem`
Identity W-map	`BusyLean7.wmap_identity`
Identity self-cast	`BusyLean7.castValue_self`
Certificate semantics	`BusyLean7.interpret_amplified`
Successor margin	`BusyLean7.next_margin`
Finite iteration bound	`BusyLean7.finite_bound`
Omega diagonal margin	`BusyLean7.omega_diagonal_margin`
Omega iteration bound	`BusyLean7.omega_bound`
Ramification base margin	`BusyLean7.ramify_margin`
Uniform final bound	`BusyLean7.final_bound`
46-fold scalar bound	`BusyLean.contender_6_tower_lt_contender_7`
Strict scalar comparison	`BusyLean.contender_6_lt_contender_7`
Eventual schedule domination	`BusyLean7.ramify_index_eventually_dominates`
Diagonal non-majorization	`BusyLean7.no_eventual_self_majorant`

All entries in this correspondence table refer to supplied source, not to completed Lean kernel checks.

References

[1] BusyLean contributors, *BusyLean: name the biggest number, in Lean*, root README at commit `86b360648163f0249eab79539438b6a83dfe8215`, retrieved October 12, 2026. Pinned README and contest rules.

[2] BusyLean contributors, *Contender 6*, `BusyLean/Contender6.lean`, same pinned commit. Pinned definition.

[3] BusyLean contributors, *Reflection over the W-language*, `BusyLean/Reflection.lean`, same pinned commit. Pinned reflection source.

[4] BusyLean contributors, *A conservative finite-type W-language and a constructive large-number entry*, proposal P5, `lean/BigNumber/Definitions.lean`, same pinned commit. Pinned language and interpreter.

[5] BusyLean contributors, *Contender 6: comparison theorems*, `BusyLean/Comparison.lean`, same pinned commit. Pinned comparison theorems.

[6] BusyLean contributors, *Review of the five incoming proposals*, `proposals/REVIEW.md`, same pinned commit. Pinned review; its ordinal calibrations are explicitly informal.

[7] BusyLean contributors, *Audit*, `test/Audit.lean`, same pinned commit. Pinned dependency and axiom audit pattern.

[8] Lean development team, *The Lean Language Reference: Recursive Definitions*, official documentation, consulted October 12, 2026. Recursive definitions and termination checking.

[9] Lean development team, *The Lean Language Reference: Inductive Types*, official documentation, consulted October 12, 2026. Inductive types and their recursors.

[10] Lean development team, *Lean 4.34.0 release*, official release record. Pinned toolchain release.